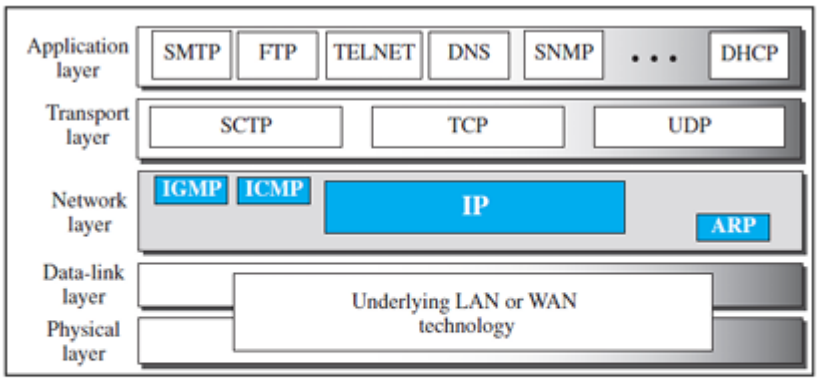
1. Give a brief introduction about IP Security

- ➔ The network layer in version 4 can be thought of as one main protocol and three auxiliary ones.
- ➔ The main protocol, Internet Protocol version 4 (IPv4), is responsible for packetizing, forwarding, and delivery of a packet at the network layer.
- ➔ The Internet Control Message Protocol version 4 (ICMPv4) helps IPv4 to handle some errors that may occur in the network-layer delivery. The Internet Group Management Protocol (IGMP) is used to help IPv4 in multicasting.
- ➔ The Address Resolution Protocol (ARP) is used to glue the network and data-link layers in mapping network-layer addresses to link-layer addresses. Figure shows the positions of these four protocols in the TCP/IP protocol suite.



- ➔ IPv4 is an unreliable datagram protocol—a best-effort delivery service. The term *best-effort* means that IPv4 packets can be corrupted, be lost, arrive out of order, or be delayed, and may create congestion for the network.
- ➔ IPv4 is also a connectionless protocol that uses the datagram approach. This means that each datagram is handled independently, and each datagram can follow a different route

2. Explain IPV6 Protocol with architecture.

The change of the IPv6 address size requires the change in the IPv4 packet format. The following shows other changes implemented in the protocol in addition to changing address size and format.

Better header format. IPv6 uses a new header format in which options are separated from the base header and inserted, when needed, between the base header and the data.

New options. IPv6 has new options to allow for additional functionalities.

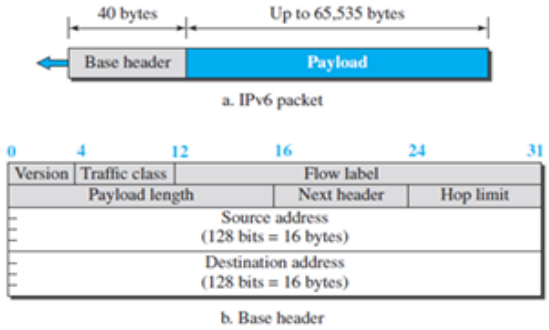
Allowance for extension. IPv6 is designed to allow the extension of the protocol if required by new technologies or applications.

Support for resource allocation. In IPv6, the type-of-service field has been removed, but two new fields, traffic class and flow label, have been added to enable the source to request special handling of the packet. This mechanism can be used to support traffic such as real-time audio and video.

Support for more security. The encryption and authentication options in IPv6 provide confidentiality and integrity of the packet.

Packet Format

The IPv6 packet is shown in Figure. Each packet is composed of a base header followed by the payload. The base header occupies 40 bytes, whereas payload can be up to 65,535 bytes of information. The description of fields follows.



Version. The 4-bit version field defines the version number of the IP. For IPv6, the value is 6.

Traffic class. The 8-bit traffic class field is used to distinguish different payloads with different delivery requirements. It replaces the *type-of-service* field in IPv4.

Flow label. The flow label is a 20-bit field that is designed to provide special handling for a particular flow of data.

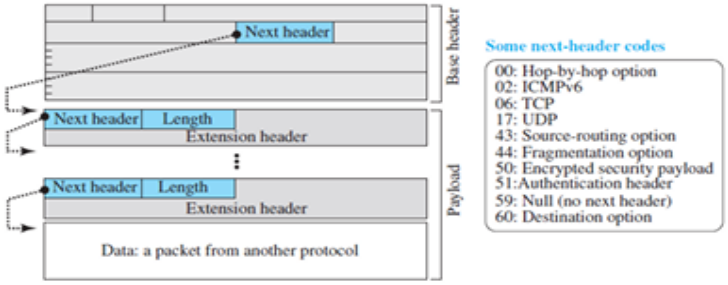
Payload length. The 2-byte payload length field defines the length of the IP datagram excluding the header.

Next header. The **next header** is an 8-bit field defining the type of the first extension header (if present) or the type of the data that follows the base header in the datagram..

Hop limit. The 8-bit hop limit field serves the same purpose as the TTL field in IPv4.

Source and destination addresses. The source address field is a 16-byte (128-bit) Internet address that identifies the original source of the datagram. The destination address field is a 16-byte (128-bit) Internet address that identifies the destination of the datagram.

Payload. Compared to IPv4, the payload field in IPv6 has a different format and meaning, as shown in Figure.



- ➔ The payload in IPv6 means a combination of zero or more extension headers (options) followed by the data from other protocols (UDP, TCP, and so on).
- ➔ In IPv6, options, which are part of the header in IPv4, are designed as extension headers. The payload can have as many extension headers as required by the situation.
- ➔ Each extension header has two mandatory fields, next header and the length, followed by information related to the particular option.
- ➔ Note that each next header field value (code) defines the type of the next header (hop-by-hop option, sourcerouting option, . . .); the last next header field defines the protocol (UDP, TCP, . . .) that is carried by the datagram.

3. Explain fragmentation in IP and explain how reassembly is performed at the destination.

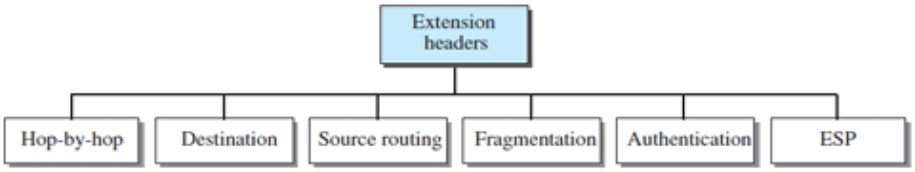
Fragmentation and Reassembly

- ➔ IPv6 datagrams can be fragmented only by the source, not by the routers; the reassembly takes place at the destination.

- ➔ The fragmentation of a packet in a router needs a lot of processing hence processing at router is not allowed.
- ➔ In IPv6, the source can check the size of the packet and make the decision to fragment the packet or not.
- ➔ When a router receives the packet, it can check the size of the packet and drop it if the size is larger than allowed by the MTU of the network ahead.
- ➔ The router then sends a packet-too-big ICMPv6 error message to inform the source.

Extension Header

An IPv6 packet is made of a base header and some extension headers. The length of the base header is fixed at 40 bytes. The base header can be followed by up to six **extension headers**, to give more functionality**.**



Hop-by-Hop Option

The *hop-by-hop option* is used when the source needs to pass information to all routers visited by the datagram. So far, only three hop by- hop options have been defined: Pad1, PadN, and jumbo payload.

Pad1. This option is 1 byte long and is designed for alignment purposes. Some options need to start at a specific bit of the 32-bit word. If an option falls short of this requirement by exactly one byte, Pad1 is added.

PadN. PadN is similar in concept to Pad1. The difference is that PadN is used when 2 or more bytes are needed for alignment.

Jumbo payload. If for any reason a longer payload(>65,535 bytes) is required, the jumbo payload option is used to define this longer length.

Destination Option

The **destination option** is used when the source needs to pass information to the destination only. Intermediate routers are not permitted access to this information.

Source Routing

The source routing extension header combines the concepts of the strict source route and the loose source route options of IPv4.

Fragmentation

In IPv6, only the original source can fragment. A source must use a **Path MTU Discovery technique** to find the smallest MTU supported by any network on the path. The source then fragments using this knowledge.

Authentication

The **authentication** extension header has a dual purpose: it validates the message sender and ensures the integrity of data.

Encrypted Security Payload

The **encrypted security payload (ESP)** is an extension that provides confidentiality and guards against eavesdropping.

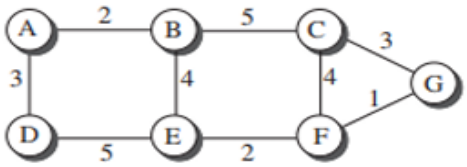
Comparison of Options between IPv4 and IPv6

The following shows a quick comparison between the options used in IPv4 and the options used in IPv6 (as extension headers).

- ➔ The no-operation and end-of-option options in IPv4 are replaced by Pad1 and PadN options in IPv6.
- ➔ The record route option is not implemented in IPv6 because it was not used.
- ➔ The timestamp option is not implemented because it was not used.
- ➔ The source route option is called the *source route extension header* in IPv6.
- ➔ The fragmentation fields in the base header section of IPv4 have moved to the fragmentation extension header in IPv6.
- ➔ The authentication extension header is new in IPv6.
- ➔ The encrypted security payload extension header is new in IPv6

4. Explain the Link State Routing Algorithm.

- ➔ Link-State (LS) routing is a method used by routers to determine the best, or least-cost, path through a network.
- ➔ In this method, each connection between routers is referred to as a **link**, and the **link-state** describes the cost of using that connection. Lower costs are preferred, while a link with an infinite cost means it is unavailable or broken.
- ➔ To create efficient routing paths, each router must have a complete map of the network, which includes the state (cost) of every link. This information is stored in the **Link-State Database (LSDB)**, which is shared across all routers.
- ➔ The LSDB is often represented as a matrix, where each value shows the cost of a link between two routers.
- ➔ With this database, each router can build the least-cost tree, ensuring it knows the shortest routes to every other router.



a. The weighted graph

	A	B	C	D	E	F	G
A	0	2	∞	3	∞	∞	∞
B	2	0	5	∞	4	∞	∞
C	∞	5	0	∞	∞	4	3
D	3	∞	∞	0	5	∞	∞
E	∞	4	∞	5	0	2	∞
F	∞	∞	4	∞	2	0	1
G	∞	∞	3	∞	∞	1	0

b. Link state database

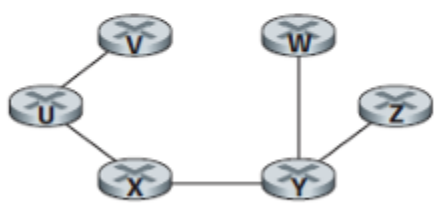
- ➔ To create the **Link-State Database (LSDB)**, routers use a process called **flooding**. Each router sends a greeting to its direct neighbors to gather two pieces of information: the neighbor's identity and the cost of the link.
- ➔ This data is packaged into a **Link-State Packet (LSP)** and sent to all neighboring routers. When a router receives an LSP, it checks if the information is new by comparing the sequence number.
- ➔ If the LSP is new, the router keeps it and sends copies to other neighbors, except the one from which it was received.
- ➔ This process repeats until all routers have the latest LSPs, allowing each router to create an identical **LSDB**, which provides a complete map of the network. This LSDB helps routers calculate the least-cost paths to all other nodes.

- The link-state routing algorithm is known as **Dijkstra's algorithm**. Dijkstra's algorithm computes the least-cost path from one node to all other nodes in the network.
- The following notations are used:
  - D(v): cost of the least-cost path from the source node to destination v as of this iteration of the algorithm.
  - p(v): previous node (neighbor of v) along the current least-cost path from the source to v.
  - N' : subset of nodes; v is in N ' if the least-cost path from the source to v is definitively known.

```
Initialization:
  N' = {u}
  for all nodes v
    if v is a neighbor of u
      then D(v) = c(u,v)
    else D(v) = ∞

Loop
  find w not in N' such that D(w) is a minimum
  add w to N'
  update D(v) for each neighbor v of w and not in N':
        D(v) = min( D(v), D(w) + c(w,v) )
  /* new cost to v is either old cost to v or known
   least path cost to w plus cost from w to v */
until N'= N
```

step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4	uxyvw					4,y
5	uxyvwz					



Destination	Link
v	(u, v)
w	(u, x)
x	(u, x)
y	(u, x)
z	(u, x)

Least cost path and forwarding table for nodule u

5. Compare RIP and OSPF with diagram.

Difference Between RIP and OSPF

Point	RIP (Routing Information Protocol)	OSPF (Open Shortest Path First)
Type of Protocol	Distance Vector Protocol	Link State Protocol
Algorithm Used	Bellman–Ford Algorithm	Dijkstra's Shortest Path First (SPF) Algorithm
Metric Used	Hop Count	Cost (based on bandwidth)
Maximum Hop Count	15 hops (16 = unreachable)	No hop limit
Convergence Speed	Slow	Very fast
Updates	Periodic updates every 30 seconds	Updates only when topology changes
Routing Table Size	Larger and less efficient	Smaller and optimized
Network Type Suitable	Small networks	Medium to large enterprise networks
Classful / Classless	RIP v1 – Classful; RIP v2 – Classless	Classless
Support for VLSM & CIDR	Only RIP v2 supports it	Fully supported
Authentication	Basic authentication	Strong authentication (MD5 etc.)
Hierarchy Support	No hierarchy	Supports hierarchical design with Areas
Resource Usage	Low CPU and memory	Higher CPU and memory
Broadcast/Multicast	RIP v1 uses broadcast; RIP v2 uses multicast	Uses multicast (224.0.0.5 and 224.0.0.6)

6. Explain multicast routing protocol.

Multicast Link State (MOSPF)

Path First (OSPF) protocol, which is used in unicast routing. It also uses the source-based tree approach to multicasting. If the internet is running a unicast link-state routing algorithm, the idea can be extended to provide a multicast link-state routing algorithm.

To extend unicasting to multicasting, each router needs to have another database, as with the case of unicast distance-vector routing, to show which interface has an active member in a particular group. Now a router goes through the following steps to forward a multicast packet received from source S and to be sent to destination G (a group of recipients):

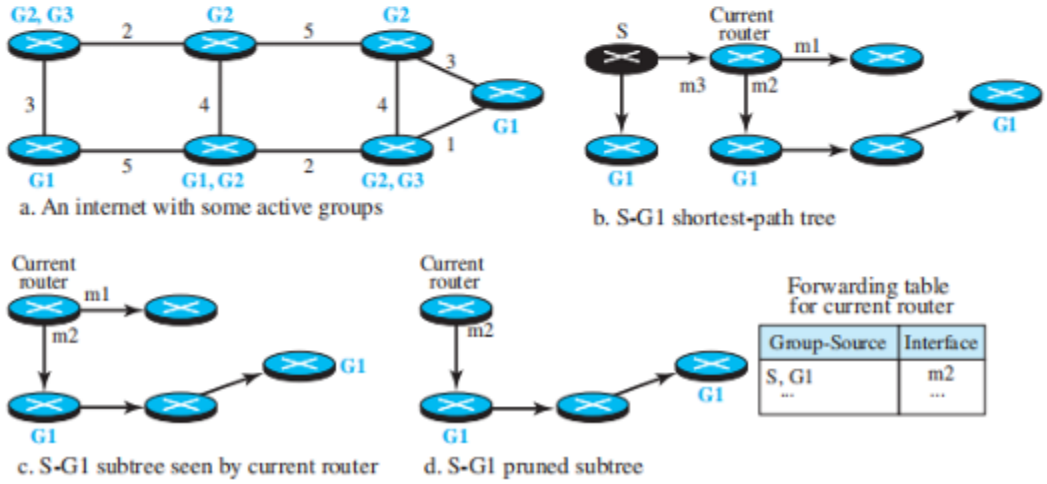
1. The router uses the Dijkstra algorithm to create a shortest-path tree with S as the root and all destinations in the internet as the leaves. Note that this shortest-path tree is different from the one the router normally uses for unicast forwarding, in which the root of the tree is the router itself. In this case, the root of the tree is the source of the packet defined in the source address of the packet. The router is capable of creating this tree because it has the LSDB, the whole topology of the internet; the Dijkstra algorithm can be used to create a tree with any root, no matter which router is using it. The point we need to remember is that the shortest-path tree created this way depends on the specific source. For each source we need to create a different tree.

2. The router finds itself in the shortest-path tree created in the first step. In other words, the router creates a shortest-path subtree with itself as the root of the subtree.

3. The shortest-path subtree is actually a broadcast subtree with the router as the root and all networks as the leaves. The router now uses a strategy similar to the one we describe in the case of DVMRP to prune the broadcast tree and to change it to a multicast tree. The IGMP protocol is used to find the information at the leaf level. MOSPF has added a new type of link state update packet that floods the membership to all routers. The router can use the information it receives in this way and prune the broadcast tree to make the multicast tree.

4. The router can now forward the received packet out of only those interfaces that correspond to the branches of the multicast tree. We need to make certain that a copy of the multicast packet reaches all networks that have active members of the group and that it does not reach those networks that do not.

Figure shows an example of using the steps to change a graph to a multicast tree. For simplicity, we have not shown the network, but we added the groups to each router. The figure shows how a source-based tree is made with the source as the root and changed to a multicast subtree with the root at the current router.



a. An internet with some active groups

b. S-G1 shortest-path tree

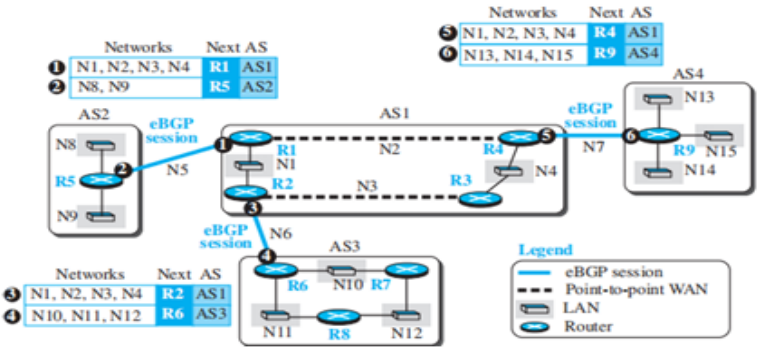
c. S-G1 subtree seen by current router

d. S-G1 pruned subtree

7. Bring out the difference between Operation of External eBGP and Internal iBGP

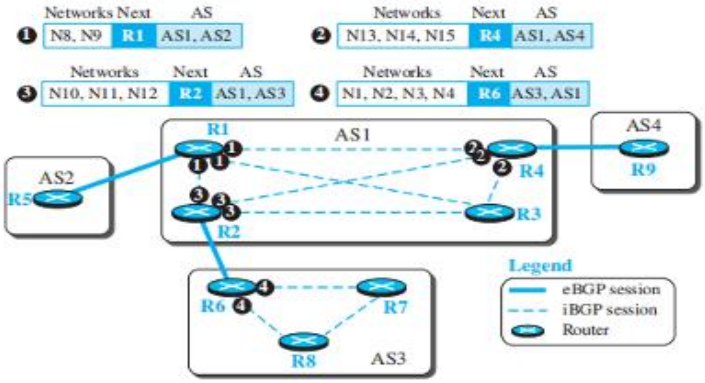
External BGP (eBGP) Operation

BGP operates over Transmission Control Protocol (TCP) connections, ensuring reliable communication between routers. The protocol defines two types of operations: external (eBGP) and internal (iBGP). eBGP sessions are established between routers belonging to different ASs, as seen in Figure. In these sessions, routers share routing information to other ASs by exchanging reachability information about network prefixes. However, this only provides visibility between the border routers, so internal routers need iBGP for full route dissemination.



Internal BGP (iBGP) Operation

Within the same AS, routers establish iBGP sessions to disseminate the routes learned from eBGP sessions. A fully connected mesh of iBGP sessions is required to prevent routing loops. Each iBGP router must have a direct session with every other iBGP router, as depicted in Figure.



This ensures that all routers inside an AS have complete knowledge of external networks. After iBGP sessions converge, all routers will have consistent routing information about the external networks, allowing them to build accurate path tables, as demonstrated in Figure.

Networks	Next	Path
N8, N9	R5	AS1, AS2
N10, N11, N12	R2	AS1, AS3
N13, N14, N15	R4	AS1, AS4

Path table for R1

Networks	Next	Path
N8, N9	R1	AS1, AS2
N10, N11, N12	R6	AS1, AS3
N13, N14, N15	R1	AS1, AS4

Path table for R2

Networks	Next	Path
N8, N9	R2	AS1, AS2
N10, N11, N12	R2	AS1, AS3
N13, N14, N15	R4	AS1, AS4

Path table for R3

Networks	Next	Path
N8, N9	R1	AS1, AS2
N10, N11, N12	R1	AS1, AS3
N13, N14, N15	R9	AS1, AS4

Path table for R4

Networks	Next	Path
N1, N2, N3, N4	R1	AS2, AS1
N10, N11, N12	R1	AS2, AS1, AS3
N13, N14, N15	R1	AS2, AS1, AS4

Path table for R5

Networks	Next	Path
N1, N2, N3, N4	R2	AS3, AS1
N8, N9	R2	AS3, AS1, AS2
N13, N14, N15	R2	AS3, AS1, AS4

Path table for R6

Networks	Next	Path
N1, N2, N3, N4	R6	AS3, AS1
N8, N9	R6	AS3, AS1, AS2
N13, N14, N15	R6	AS3, AS1, AS4

Path table for R7

Networks	Next	Path
N1, N2, N3, N4	R6	AS3, AS1
N8, N9	R6	AS3, AS1, AS2
N13, N14, N15	R6	AS3, AS1, AS4

Path table for R8

Networks	Next	Path
N1, N2, N3, N4	R4	AS4, AS1
N8, N9	R4	AS4, AS1, AS2
N10, N11, N12	R4	AS4, AS1, AS3

Path table for R9

8. Explain Path Vector Routing with Algorithm or a node.

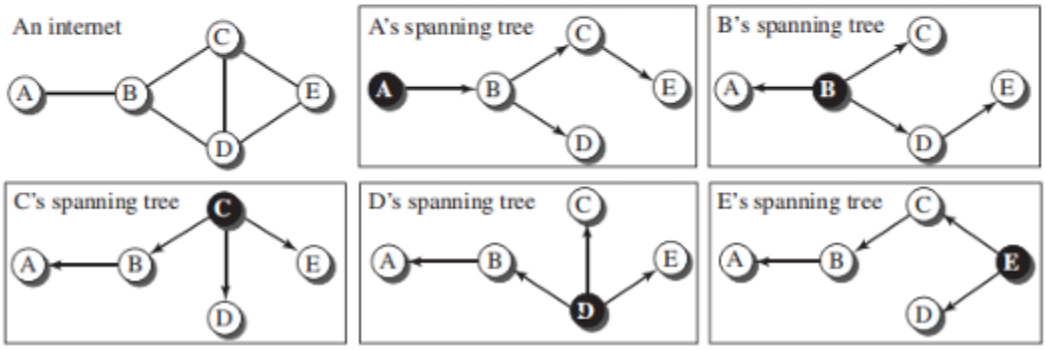
Path-Vector Routing

The **link-state (LS)** and **distance-vector (DV)** routing focus on finding the least-cost path, but there are situations where this goal isn't the priority. For example, a sender may want to avoid certain routers due to security concerns or competition, but least-cost routing doesn't allow for such policies. These methods only focus on minimizing cost, meaning packets could still pass through undesired routers if they are part of the least-cost path. In some cases, such as for security or policy reasons, the sender may prioritize **reachability** over cost, ensuring that packets reach their destination efficiently without considering the route's cost.

Path-vector (PV) routing is a routing algorithm designed to address the limitations of **link-state (LS)** and **distance-vector (DV)** routing, which focus on least-cost paths. Unlike those methods, PV routing allows the source to control the route based on policies, such as avoiding certain routers for security or business reasons. While primarily used for routing between ISPs, PV routing offers flexibility by letting the source determine the best route according to its own policies rather than cost.

Spanning Trees

In **path-vector routing**, the route from a source to all destinations is determined by the **best spanning tree**, but unlike the least-cost tree, the best spanning tree is chosen based on the source's policies. If multiple routes exist, the source can pick the route that aligns with its preferences, which may include avoiding certain nodes or minimizing the number of nodes visited. For example, in a network of five nodes, sources like **A** and **E** may create spanning trees that avoid node **D** as an intermediary, while **B** may avoid node **C**. These policies allow the source to shape the path according to its specific needs.



Creation of Spanning Trees

Path-vector routing, like distance-vector routing, is an asynchronous and distributed routing algorithm. The spanning trees are made, gradually and asynchronously, by each node. When a node is booted, it creates a path vector based on the information it can obtain about its immediate

neighbour. A node sends greeting messages to its immediate neighbours to collect these pieces of information.

Each node, after the creation of the initial path vector, sends it to all its immediate neighbours. Each node, when it receives a path vector from a neighbour, updates its path vector using an equation similar to the Bellman-Ford, but applying its own policy instead of looking for the least cost. We can define this equation as

$$\text{Path}(x, y) = \text{best}\ \{\text{Path}(x, y), [(x + \text{Path}(v, y)]\} \quad \text{for all } v\text{'s in the internet.}$$

```
Path_Vector_Routing ( )
{
    // Initialization
    for (y = 1 to N)
    {
        if (y is myself)
            Path [y] = myself
        else if (y is a neighbor)
            Path [y] = myself + neighbor node
        else
            Path [y] = empty
    }
    Send vector {Path[1], Path[2], …, Path[y]} to all neighbors
    // Update
    repeat (forever)
    {
        wait (for a vector Path_w from a neighbor w)
        for (y = 1 to N)
        {
            if (Path_w includes myself)
                discard the path                    // Avoid any loop
            else
                Path[y] = best {Path[y], (myself + Path_w[y])}
        }
        If (there is a change in the vector)
            Send vector {Path[1], Path[2], …, Path[y]} to all neighbors
    }
} // End of Path Vector
```

9. How IPv6 changes Differ from IPv4

THE IPv6 PROTOCOL

The change of the IPv6 address size requires the change in the IPv4 packet format. The following shows other changes implemented in the protocol in addition to changing address size and format.

Better header format. IPv6 uses a new header format in which options are separated from the base header and inserted, when needed, between the base header and the data.

New options. IPv6 has new options to allow for additional functionalities.

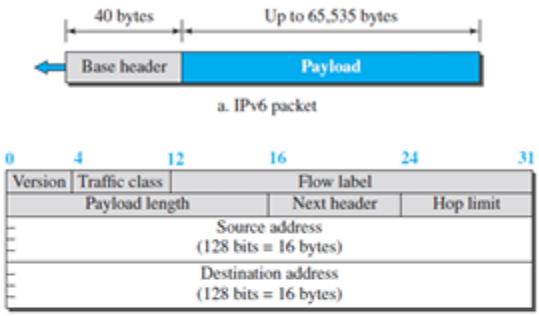
Allowance for extension. IPv6 is designed to allow the extension of the protocol if required by new technologies or applications.

Support for resource allocation. In IPv6, the type-of-service field has been removed, but two new fields, traffic class and flow label, have been added to enable the source to request special handling of the packet. This mechanism can be used to support traffic such as real-time audio and video.

Support for more security. The encryption and authentication options in IPv6 provide confidentiality and integrity of the packet.

Packet Format

The IPv6 packet is shown in Figure. Each packet is composed of a base header followed by the payload. The base header occupies 40 bytes, whereas payload can be up to 65,535 bytes of information. The description of fields follows.



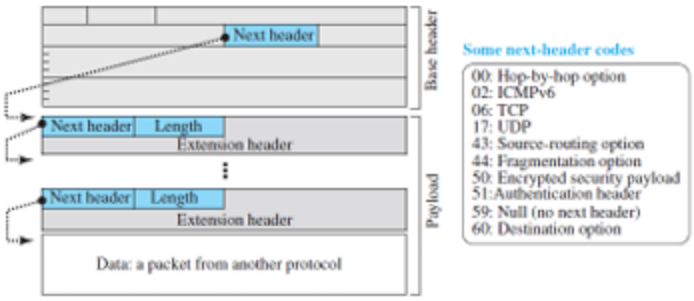
Version. The 4-bit version field defines the version number of the IP. For IPv6, the value is 6.

Traffic class. The 8-bit traffic class field is used to distinguish different payloads with different delivery requirements. It replaces the *type-of-service* field in IPv4.

Flow label. The flow label is a 20-bit field that is designed to provide special handling for a particular flow of data.

Payload length. The 2-byte payload length field defines the length of the IP datagram excluding the header.

Next header. The **next header** is an 8-bit field defining the type of the first extension header (if present) or the type of the data that follows the base header in the datagram..
Hop limit. The 8-bit hop limit field serves the same purpose as the TTL field in IPv4.
Source and destination addresses. The source address field is a 16-byte (128-bit) Internet address that identifies the original source of the datagram. The destination address field is a 16-byte (128-bit) Internet address that identifies the destination of the datagram.
Payload. Compared to IPv4, the payload field in IPv6 has a different format and meaning, as shown in Figure.



- ➔ The payload in IPv6 means a combination of zero or more extension headers (options) followed by the data from other protocols (UDP, TCP, and so on).
- ➔ In IPv6, options, which are part of the header in IPv4, are designed as extension headers. The payload can have as many extension headers as required by the situation.
- ➔ Each extension header has two mandatory fields, next header and the length, followed by information related to the particular option.
- ➔ Note that each next header field value (code) defines the type of the next header (hop-by-hop option, sourcerouting option, . . .); the last next header field defines the protocol (UDP, TCP, . . .) that is carried by the datagram.

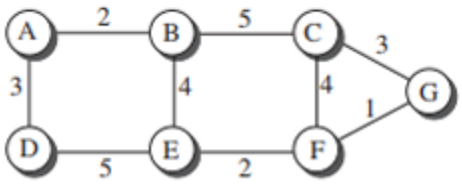
Concept of Flow and Priority in IPv6

- ➔ In version 6, the flow label has been added to the format of the IPv6 datagram to allow us to use IPv6 as a connection-oriented protocol.
- ➔ To a router, a flow is a sequence of packets that share the same characteristics, such as traveling the same path, using the same resources, having the same kind of security, and so on.
- ➔ A router that supports the handling of flow labels has a flow label table. The table has an entry for each active flow label; each entry defines the services required by the corresponding flow label.
- ➔ In its simplest form, a flow label can be used to speed up the processing of a packet by a router.

10. Explain Dijkstra Algorithm, how it is impact in Link State Routing

Link-State Routing

- ➔ Link-State (LS) routing is a method used by routers to determine the best, or least-cost, path through a network.
- ➔ In this method, each connection between routers is referred to as a **link**, and the **link-state** describes the cost of using that connection. Lower costs are preferred, while a link with an infinite cost means it is unavailable or broken.
- ➔ To create efficient routing paths, each router must have a complete map of the network, which includes the state (cost) of every link. This information is stored in the **Link-State Database (LSDB)**, which is shared across all routers.
- ➔ The LSDB is often represented as a matrix, where each value shows the cost of a link between two routers.
- ➔ With this database, each router can build the least-cost tree, ensuring it knows the shortest routes to every other router.



a. The weighted graph

	A	B	C	D	E	F	G
A	0	2	∞	3	∞	∞	∞
B	2	0	5	∞	4	∞	∞
C	∞	5	0	∞	∞	4	3
D	3	∞	∞	0	5	∞	∞
E	∞	4	∞	5	0	2	∞
F	∞	∞	4	∞	2	0	1
G	∞	∞	3	∞	∞	1	0

b. Link state database

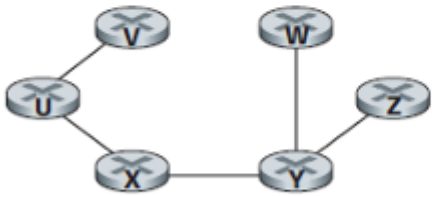
- ➔ To create the **Link-State Database (LSDB)**, routers use a process called **flooding**. Each router sends a greeting to its direct neighbors to gather two pieces of information: the neighbor's identity and the cost of the link.
- ➔ This data is packaged into a **Link-State Packet (LSP)** and sent to all neighboring routers. When a router receives an LSP, it checks if the information is new by comparing the sequence number.
- ➔ If the LSP is new, the router keeps it and sends copies to other neighbors, except the one from which it was received.
- ➔ This process repeats until all routers have the latest LSPs, allowing each router to create an identical **LSDB**, which provides a complete map of the network. This LSDB helps routers calculate the least-cost paths to all other nodes.

- ➔ The link-state routing algorithm is known as **Dijkstra's algorithm**. Dijkstra's algorithm computes the least-cost path from one node to all other nodes in the network.
- ➔ The following notations are used:
  - D(v): cost of the least-cost path from the source node to destination v as of this iteration of the algorithm.
  - p(v): previous node (neighbor of v) along the current least-cost path from the source to v.
  - N' : subset of nodes; v is in N ' if the least-cost path from the source to v is definitively known.

```
Initialization:
  N' = {u}
  for all nodes v
    if v is a neighbor of u
      then D(v) = c(u,v)
    else D(v) = ∞

Loop
  find w not in N' such that D(w) is a minimum
  add w to N'
  update D(v) for each neighbor v of w and not in N':
        D(v) = min( D(v), D(w) + c(w,v) )
  /* new cost to v is either old cost to v or known
   least path cost to w plus cost from w to v */
until N'= N
```

step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4	uxyvw					4,y
5	uxyvwz					



Destination	Link
v	(u, v)
w	(u, x)
x	(u, x)
y	(u, x)
z	(u, x)

Least cost path and forwarding table for nodule u